



IBM Developer
SKILLS NETWORK

Winning Space Race with Data Science

Nov Larsley F. Salvador
July 1, 2026



Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

Summary of Methodologies

Historical launch data were collected through both SpaceX's REST API and web scraping using BeautifulSoup. The collected data were cleaned, transformed, and prepared through data wrangling techniques to improve quality and consistency. Exploratory Data Analysis (EDA) was then conducted using Python visualizations and SQL queries to uncover trends, relationships, and factors influencing landing outcomes. Geographic analysis was performed with Folium to visualize launch site locations and nearby infrastructure, while an interactive dashboard was developed using. Finally, multiple machine learning classification models were trained and evaluated using a preprocessing pipeline, cross-validation, and hyperparameter tuning to identify the best-performing model for predicting landing success.

Summary of Results

- Successfully integrated launch data from both the SpaceX API and web scraping into a unified dataset, cleaned and transformed the data to create a high-quality dataset suitable for analysis and modeling.
- Identified important patterns and relationships between launch success and factors such as launch site, payload mass, booster version, and flight history through EDA and SQL analysis.
- Created geographic visualizations that highlight launch locations and nearby infrastructure using Folium maps.
- Built and compared several machine learning classification models using GridSearchCV achieving the best overall performance.

Introduction

Project Background and Context

SpaceX has significantly reduced the cost of space launches by developing reusable **Falcon 9** rockets. A key factor in lowering launch costs is the successful recovery and reuse of the rocket's first stage. Being able to predict whether the first-stage landing will be successful is valuable for estimating launch costs, assessing mission feasibility, and supporting decision-making for commercial and research organizations. This project applies the data science lifecycle—including data collection, preprocessing, exploratory data analysis, visualization, geospatial analysis, and machine learning—to analyze historical Falcon 9 launch data and develop a predictive model for first-stage landing success.

Problems to Find Answers

1. Which factors have the greatest influence on Falcon 9 first-stage landing success?
2. How do launch site, payload mass, booster version, orbit, and flight history affect landing outcomes?
3. What patterns can be identified through data visualization, SQL analysis, and geospatial exploration?
4. Can historical launch data be used to accurately predict whether a Falcon 9 first-stage landing will be successful?
5. Which machine learning model provides the best predictive performance for this classification problem?



Section 1

Methodology

Methodology

Executive Summary

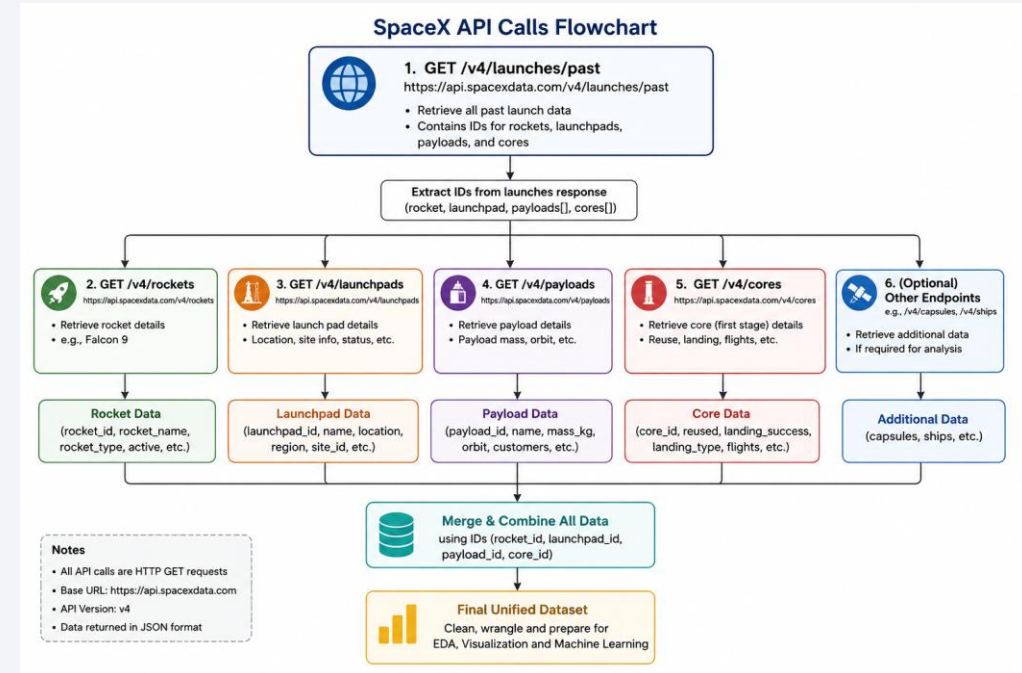
- Data collection methodology:
 - Data were collected through both SpaceX's REST API and web scraping using BeautifulSoup.
- Perform data wrangling
 - The collected data were merged, cleaned, transformed, and feature-engineered to produce a structured dataset
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
 - Classification models were built using a machine learning pipeline, optimized through GridSearchCV with cross-validation, and evaluated using accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices to identify the best-performing model.

Data Collection

Historical Falcon 9 launch data were collected from the **SpaceX REST API** and supplemented with booster information extracted through web scraping using **BeautifulSoup**. The two data sources were combined to create a comprehensive dataset containing launch details, payload information, booster versions, launch sites, and mission outcomes.

Data Collection - SpaceX API

- Send HTTP GET requests to the SpaceX REST API
- Retrieve historical Falcon 9 launch data
- Extract related resource IDs (rocket, payload, launchpad, core)
- Request additional details from linked API endpoints
- Parse JSON responses
- Select relevant launch attributes
- Combine data from multiple API endpoints
- Store results in a Pandas DataFrame



Data Collection - Scraping

- Send HTTP request to the webpage
- Parse HTML using **BeautifulSoup**
- Locate launch table elements
- Extract launch details from table rows
- Clean and format extracted text
- Convert extracted data into a Pandas DataFrame
- Export structured dataset for further processing



https://github.com/nov-salvador/SpaceX_Landing

Data Wrangling

- Merge API and web-scraped datasets
- Filter Falcon 9 launches
- Remove irrelevant features
- Handle missing values
- Clean inconsistent records
- Convert data types
- Feature extraction
- Create landing outcome (**Class**)
- Export cleaned dataset

https://github.com/nov-salvador/SpaceX_Landing



EDA with Data Visualization

1. Flight Number vs. Launch Site (Catplot)

- Compare landing outcomes across launch sites.
- Observe trends as flight numbers increase.

2. Payload vs. Launch Site (Catplot)

- Compare payload mass across launch sites.
- Examine its relationship with landing success.

3. Success Rate of Each Orbit Type (Bar Chart)

- Compare landing success rates among orbit types.
- Identify the best-performing orbits.

4. Flight Number vs. Orbit Type (Catplot)

- Analyze landing outcomes by orbit type.
- Observe trends with increasing flight numbers.

5. Payload vs. Orbit Type (Catplot)

- Explore the relationship between payload mass and orbit type.
- Assess its impact on landing success.

6. Launch Success Yearly Trend (Line Chart)

- Visualize changes in landing success over time.
- Identify long-term trends and improvements.

https://github.com/nov-salvador/SpaceX_Landing

EDA with SQL

- `select distinct "Launch_Site" from SPACEXTABLE`
- `select * from SPACEXTABLE where Launch_Site like "CCA%" limit 5`
- `select Customer, sum(PAYLOAD_MASS__KG_) from SPACEXTABLE WHERE Customer = 'NASA (CRS)'`
- `select Booster_Version, avg(PAYLOAD_MASS__KG_) as 'average payload mass' from SPACEXTABLE
where Booster_Version = 'F9 v1.1'`
- `select Min(Date), Landing_Outcome from SPACEXTABLE where Landing_Outcome = 'Success (ground pad)'`
- `select Booster_Version, Landing_Outcome, PAYLOAD_MASS__KG_ from SPACEXTABLE where Landing_Outcome = 'Success (drone ship)' and PAYLOAD_MASS__KG_ > 4000 and PAYLOAD_MASS__KG_ < 6000`
- `select case when Mission_Outcome like 'Success%' then 'Success' when Mission_Outcome like 'Failure%' then 'Failure' end as Outcome, count(*) as total from SPACEXTABLE Group by Outcome`
- `select Booster_Version, PAYLOAD_MASS__KG_ from SPACEXTABLE where PAYLOAD_MASS__KG_ = (select max(PAYLOAD_MASS__KG_) from SPACEXTABLE)`
- `select Landing_Outcome, count(*) as Total from SPACEXTABLE where Date between '2010-06-04' and '2017-03-20' group by Landing_Outcome order by Total desc`

https://github.com/nov-salvador/SpaceX_Landing

Build an Interactive Map with Folium

- **Markers**

- Display launch site locations.
- Identify each SpaceX launch complex.
- To accurately locate each SpaceX launch site on the map.

- **Circle Markers**

- Highlight launch sites.
- Differentiate sites using colors and labels.
- To emphasize launch site positions and improve visualization.

- **Distance Lines (Polylines)**

- Connect launch sites to nearby infrastructure.
- Visualize measured distances.
- To emphasize launch site positions and improve visualization

- **Distance Markers**

- Display the calculated distance values.
- Indicate proximity to surrounding features.
- To quantify distances and assess whether surrounding features may affect launch safety and site suitability.

https://github.com/nov-salvador/SpaceX_Landing

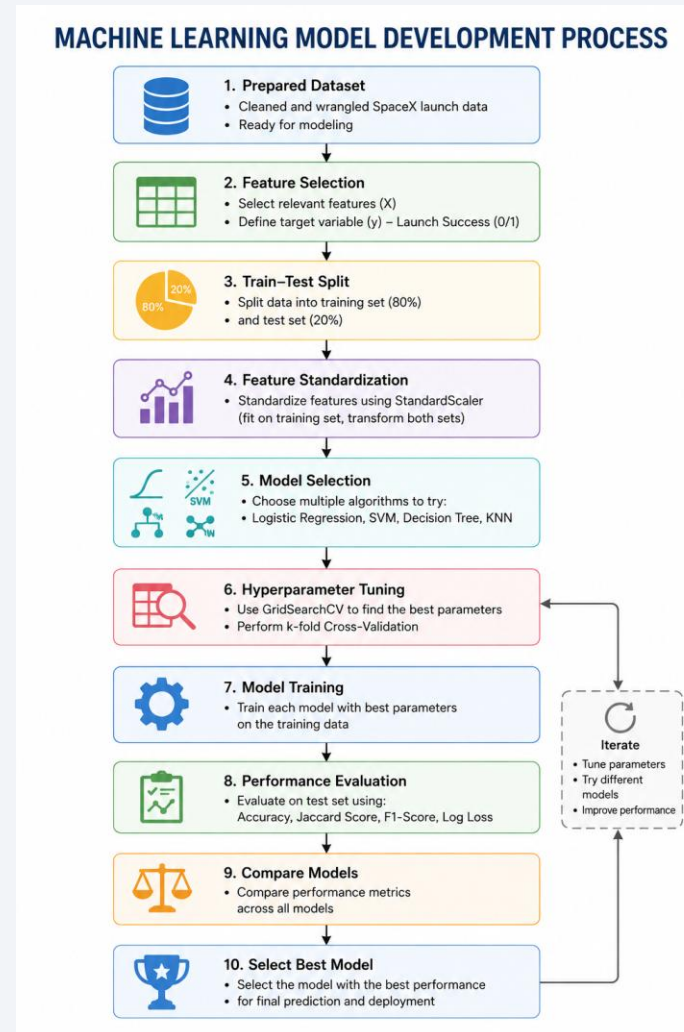
Build a Dashboard with Plotly Dash

- **Launch Site Dropdown**
 - Filter data by specific launch site or all sites.
 - Enable interactive comparison between launch locations.
 - Allows users to explore results for individual launch sites or the complete dataset.
- **Payload Range Slider**
 - Filter launches based on payload mass.
 - Analyze the effect of payload range on landing success.
 - Enables dynamic filtering to examine how payload mass influences landing success.
- **Pie Chart**
 - Display landing success distribution.
 - Compare success counts or proportions by launch site.
 - Provides a quick comparison of launch success across sites.
- **Scatter Plot**
 - Show the relationship between payload mass and landing success.
 - Differentiate launches by booster version using color.
 - Reveals patterns and relationships between payload mass, booster version, and landing outcomes.

Predictive Analysis (Classification)

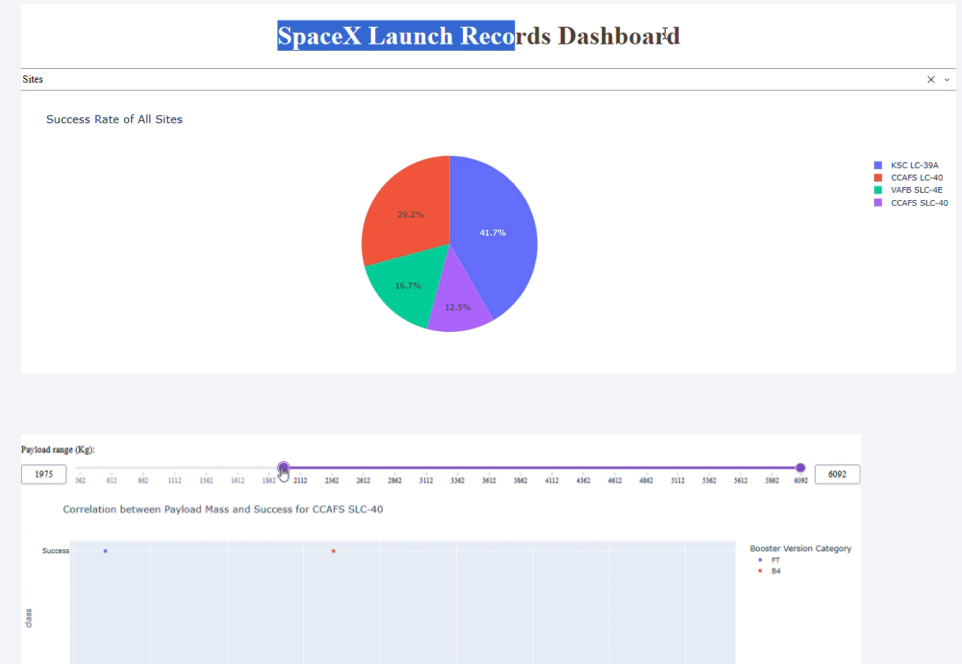
- Feature selection
- Train-test split
- Feature standardization
- Model selection
- Hyperparameter tuning (GridSearchCV)
- Cross-validation
- Model training
- Performance evaluation
- Best model selection

https://github.com/nov-salvador/SpaceX_Landing

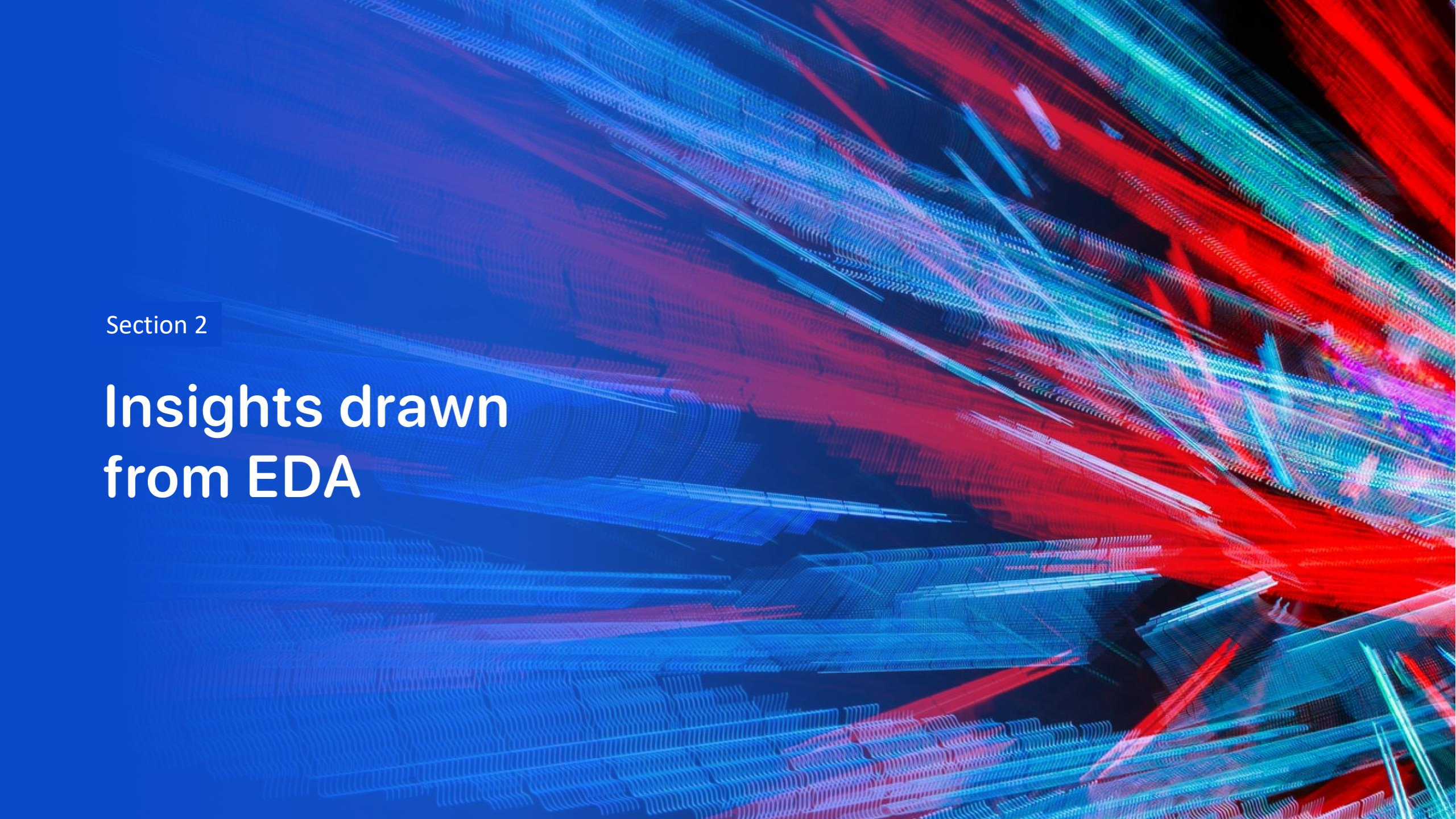


Results

- Payload Vs. Launch Site scatter point chart you will find for the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000)
- LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.
- With heavy payloads the successful landing or positive landing rate are more for Polar,LEO and ISS.
- Success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.



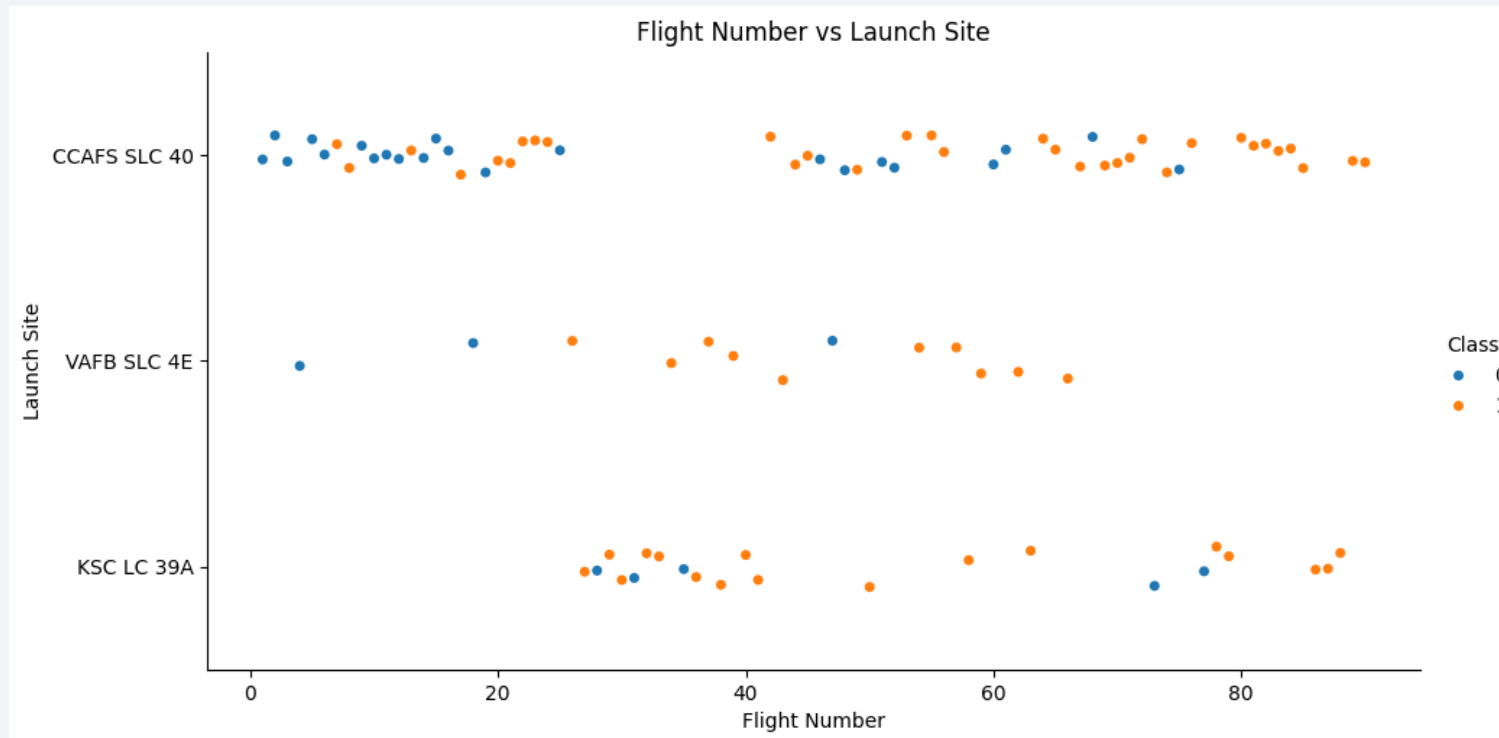
https://github.com/nov-salvador/SpaceX_Landing

The background of the slide is an abstract composition. It features a dark blue base color. Overlaid on this are numerous diagonal streaks in shades of red and cyan. A faint, light blue grid pattern is also visible, particularly in the upper right quadrant. The overall effect is dynamic and technological.

Section 2

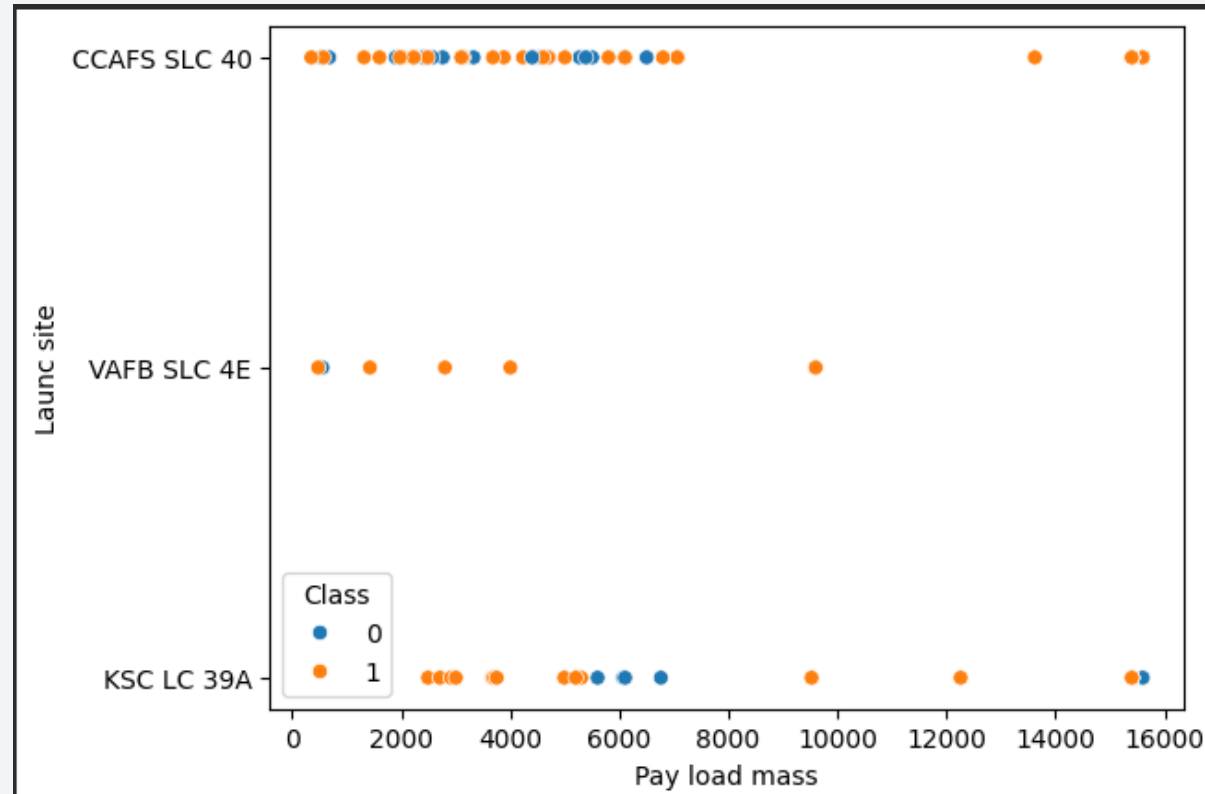
Insights drawn from EDA

Flight Number vs. Launch Site



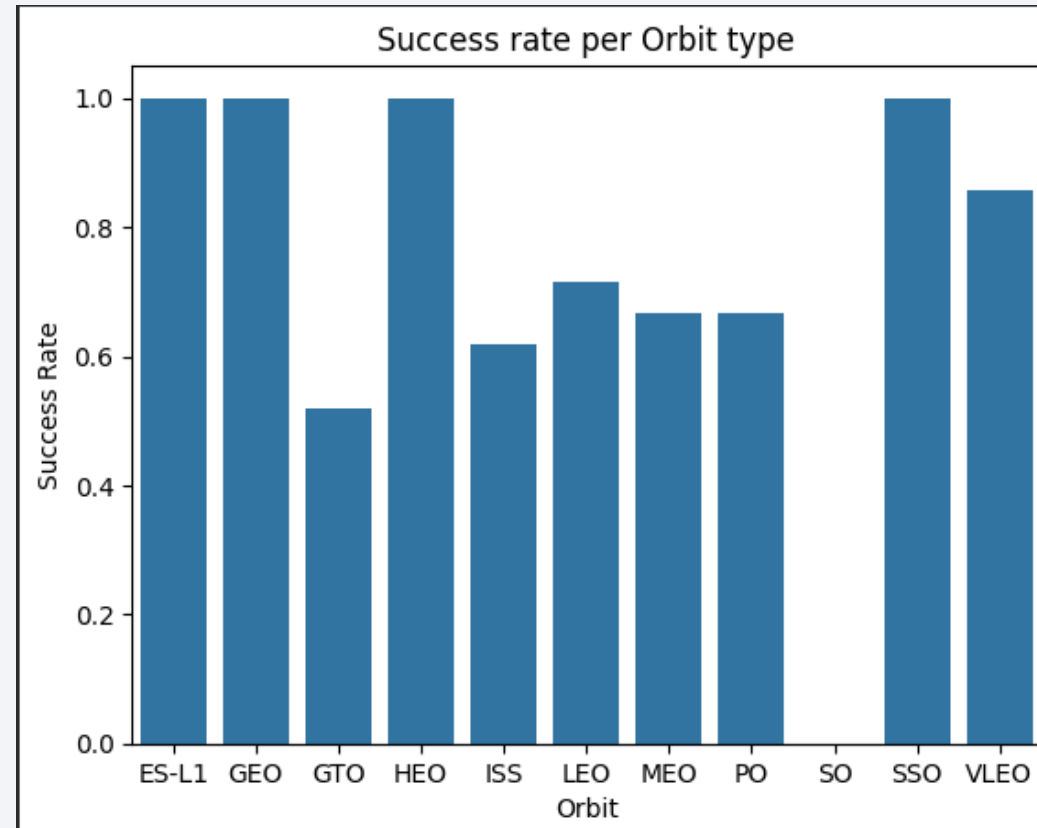
The Flight Number vs. Launch Site catplot shows that landing success generally improves as flight numbers increase, indicating that SpaceX achieved higher success rates over time across its launch sites.

Payload vs. Launch Site



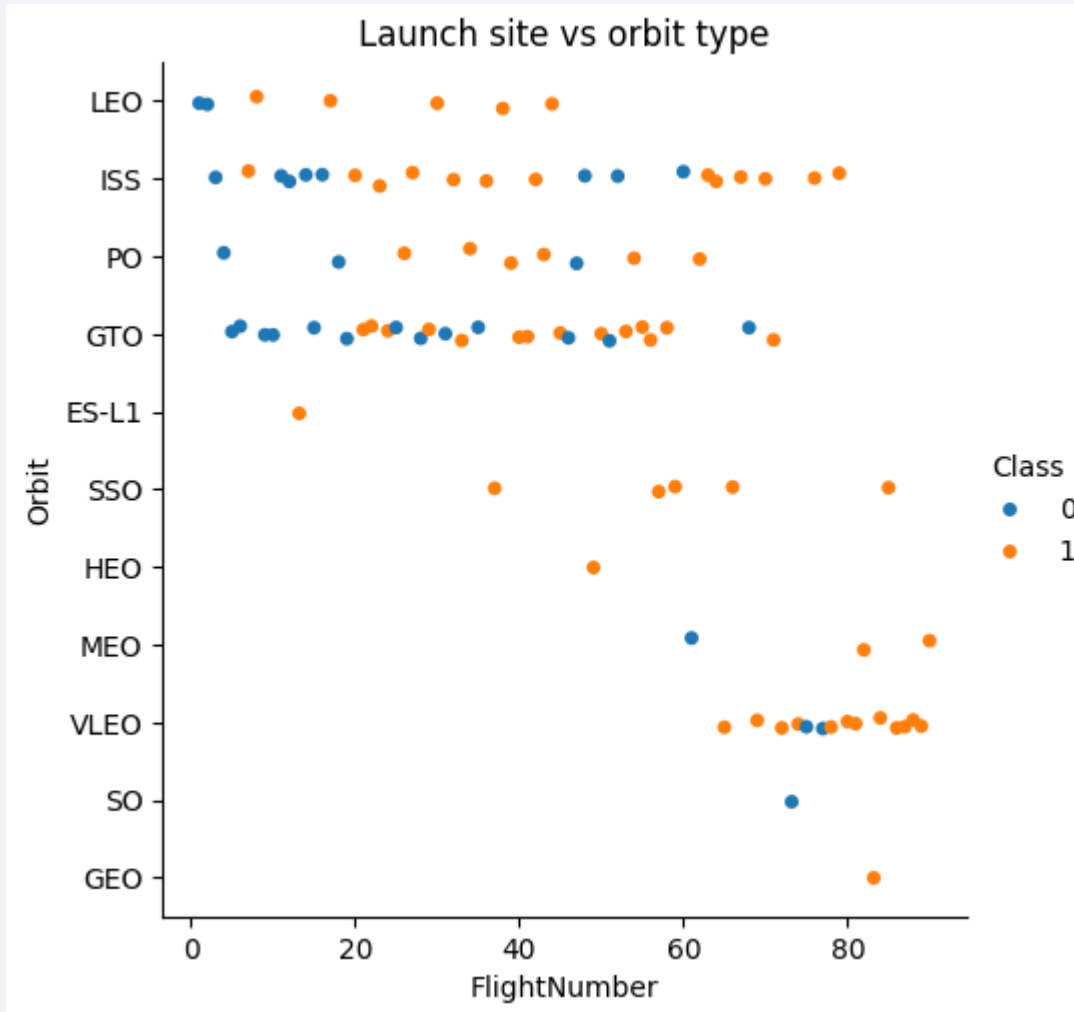
The Payload Mass vs. Launch Site catplot shows that landing success varies with payload mass across different launch sites, indicating that both payload and launch location influence mission outcomes. The VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).

Success Rate vs. Orbit Type



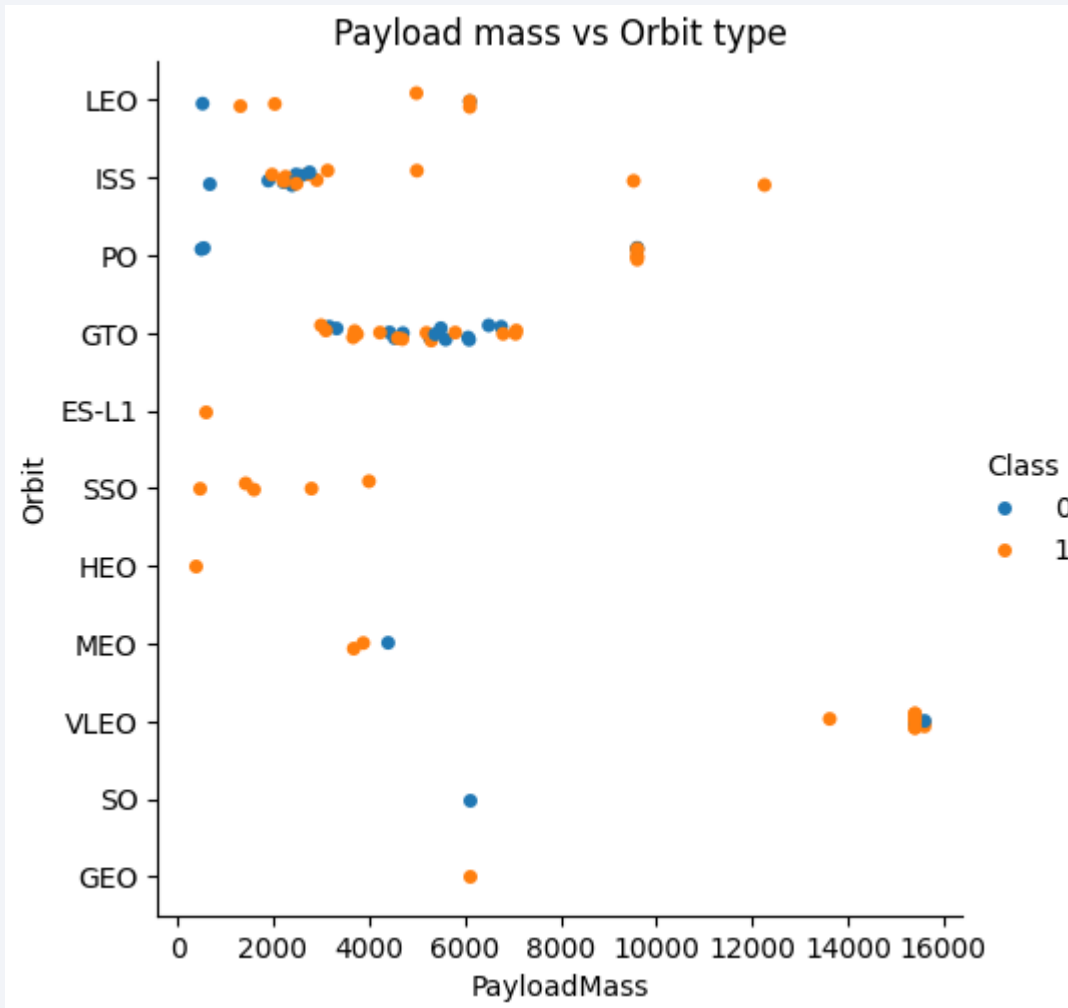
The Success Rate vs. Orbit Type bar chart shows that landing success varies across orbit types, indicating that mission orbit is an important factor influencing first-stage landing outcomes.

Flight Number vs. Orbit Type



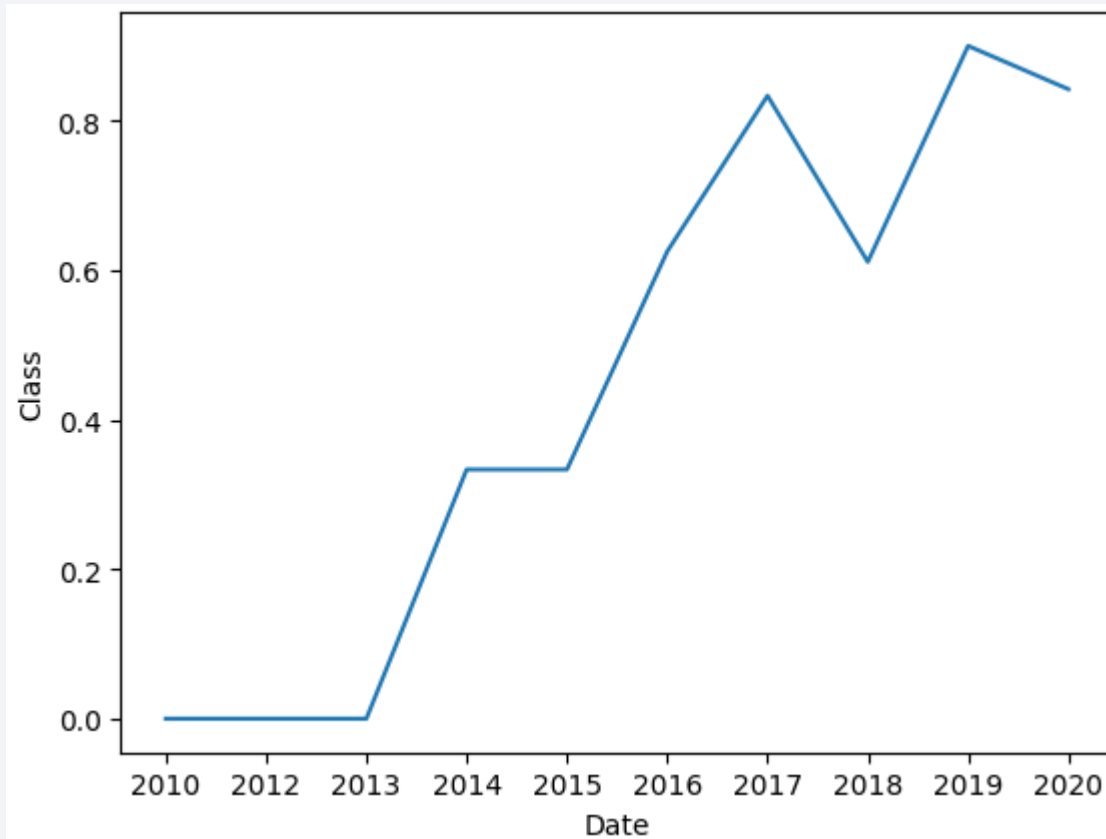
The Flight Number vs. Orbit Type catplot shows that landing success generally improves with increasing flight numbers across different orbit types. The LEO orbit the Success appears related to the number of flights; on the other hand, there seems to be no relationship between flight number when in GTO orbit.

Payload vs. Orbit Type



The Payload Mass vs. Orbit Type catplot shows how landing success varies with payload mass across different orbit types, indicating that both mission orbit and payload influence landing outcomes. With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.

Launch Success Yearly Trend



You can observe that the success rate since 2013 kept increasing till 2017 (stable in 2014) and after 2015 it started increasing.

All Launch Site Names

- Find the names of the unique launch sites

It displays the distinct launch site

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

- Find 5 records where launch sites begin with `CCA`

Using 'where' and 'like' and '%' as wildcharacter

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS_KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt

Total Payload Mass

- Calculate the total payload carried by boosters from NASA

Using 'where' to filter and sum() function on the payload mass

Customer	sum(PAYLOAD_MASS_KG_)
NASA (CRS)	45596

Average Payload Mass by F9 v1.1

- Calculate the average payload mass carried by booster version F9 v1.1
- Using 'where' to filter and avg() function on payload mass

Booster_Version	average payload mass
F9 v1.1	2928.4

First Successful Ground Landing Date

- Find the dates of the first successful landing outcome on ground pad
- Using 'where' to filter and min() function on the date

Min(Date)	Landing_Outcome
2015-12-22	Success (ground pad)

Successful Drone Ship Landing with Payload between 4000 and 6000

- List the names of boosters which have successfully landed on drone ship and had payload mass greater than 4000 but less than 6000
- Using 'where' to filter the 3 conditions and selecting required columns

Booster_Version	Landing_Outcome	PAYLOAD_MASS_KG_
F9 FT B1022	Success (drone ship)	4696
F9 FT B1026	Success (drone ship)	4600
F9 FT B1021.2	Success (drone ship)	5300
F9 FT B1031.2	Success (drone ship)	5200

Total Number of Successful and Failure Mission Outcomes

- Calculate the total number of successful and failure mission outcomes
- Check if it's a success or failure then grouping them and apply count()

Outcome	total
Failure	1
Success	100

Boosters Carried Maximum Payload

- List the names of the booster which have carried the maximum payload mass
- Get first the max() of payload then use it to the condition in 'where'

Booster_Version	PAYLOAD_MASS_KG_
F9 B5 B1048.4	15600
F9 B5 B1049.4	15600
F9 B5 B1051.3	15600
F9 B5 B1056.4	15600
F9 B5 B1048.5	15600
F9 B5 B1051.4	15600
F9 B5 B1049.5	15600
F9 B5 B1060.2	15600
F9 B5 B1058.3	15600
F9 B5 B1051.6	15600
F9 B5 B1060.3	15600
F9 B5 B1049.7	15600

2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015
- Filter failure and convert the months into names

Date	Month	Landing_Outcome	Booster_Version	Launch_Site
2015-01-10	January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
2015-04-14	April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Rank the count of landing outcomes (such as Failure (drone ship) or Success (ground pad)) between the date 2010-06-04 and 2017-03-20, in descending order
- Data between the dates, grouped and order by desc

Landing_Outcome	Total
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

A satellite view of Earth from space, showing the curvature of the planet and city lights at night. The image is a composite of a dark blue sky with stars and a view of the Earth's surface from space. The Earth's surface is mostly dark blue, with a thin layer of white clouds. A curved horizon line separates the dark sky from the Earth's surface. On the right side, there are numerous bright yellow and orange lights, representing city lights at night. The lights are concentrated in a few areas, with a large, bright cluster on the right side. The overall image has a high-contrast, high-resolution appearance, typical of satellite imagery.

Section 3

Launch Sites Proximities Analysis

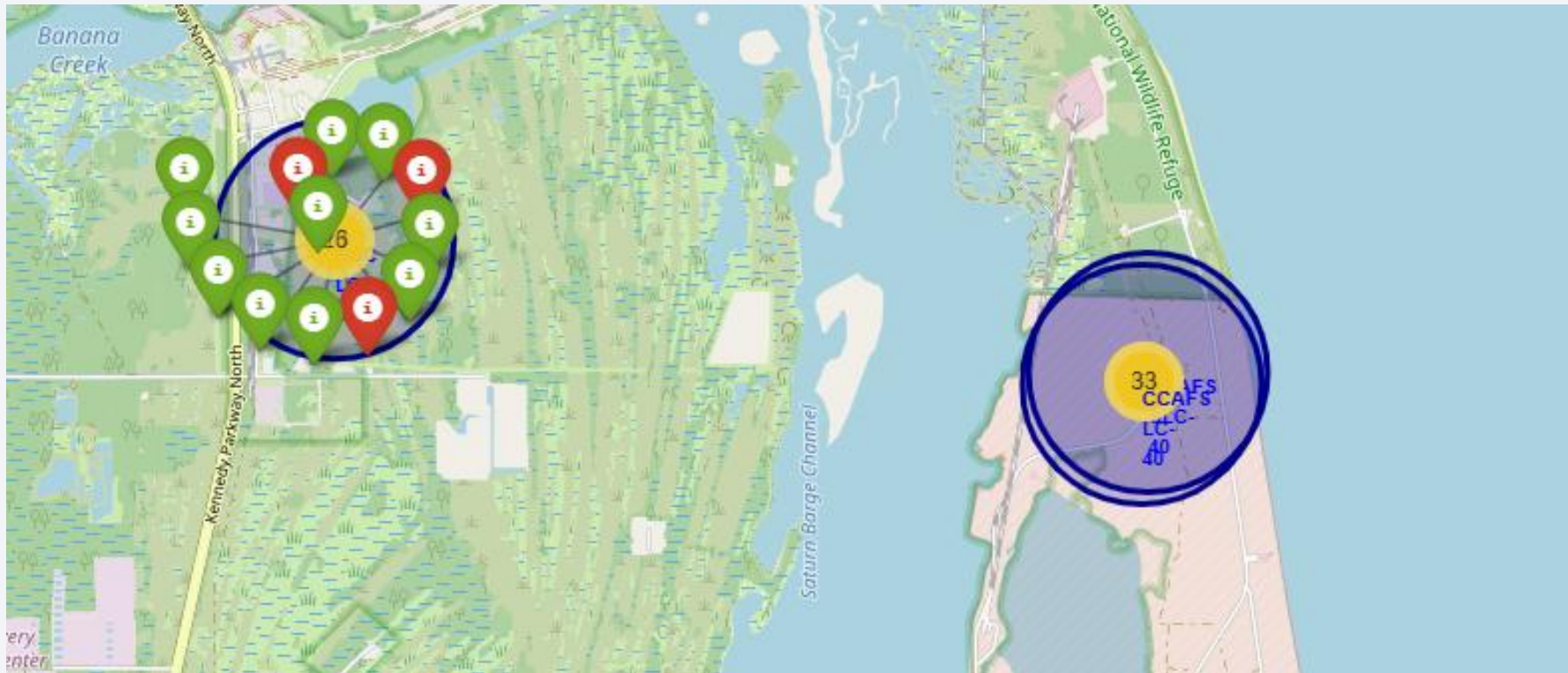
All Launch Sites

- The locations are encircled and labeled by their name



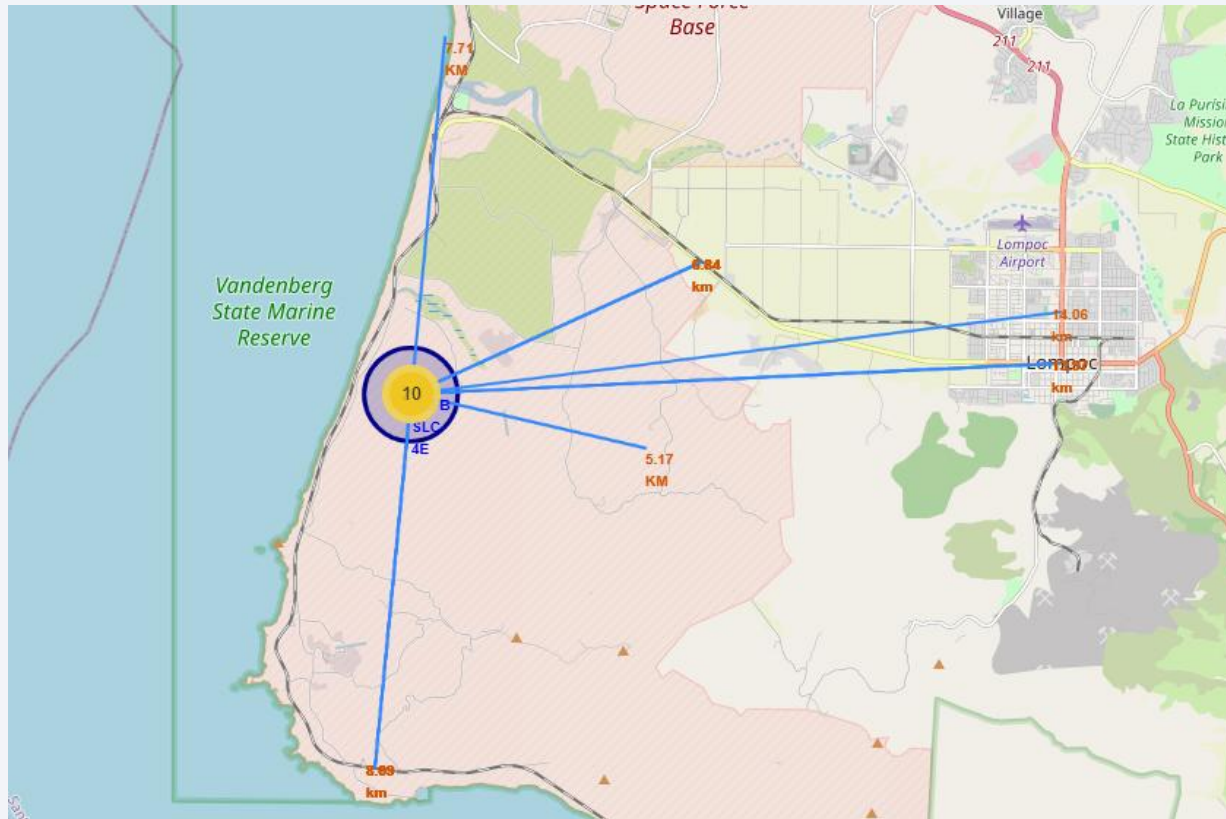
Launch Outcomes on All Sites

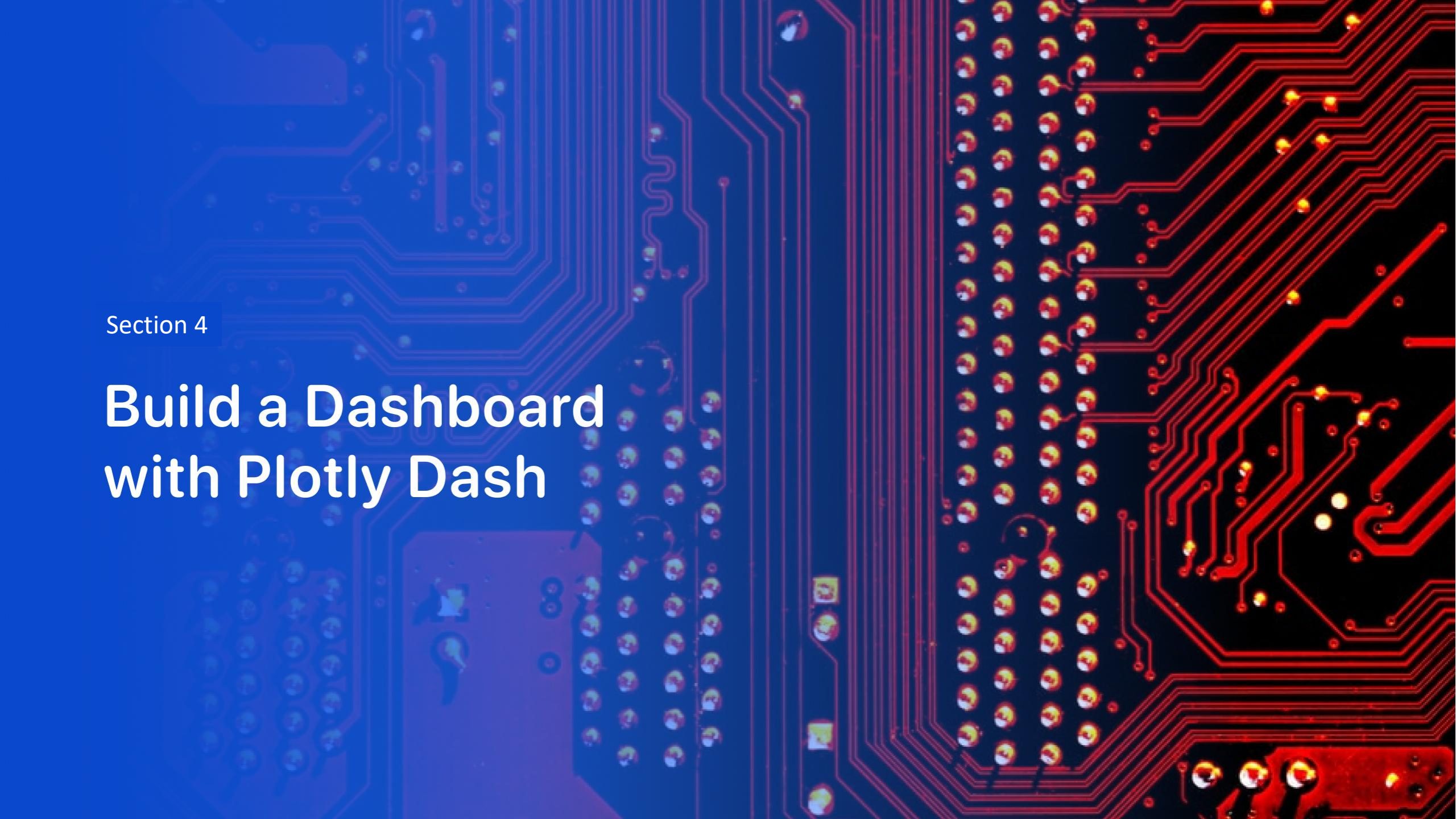
- Clusters of all color-labeled outcome on each launch site



Distance of Proximities to a Launch Site

- It shows the distance of a certain launch site to its proximities such as railway, highway, coastline and displayed the distance in kilometers



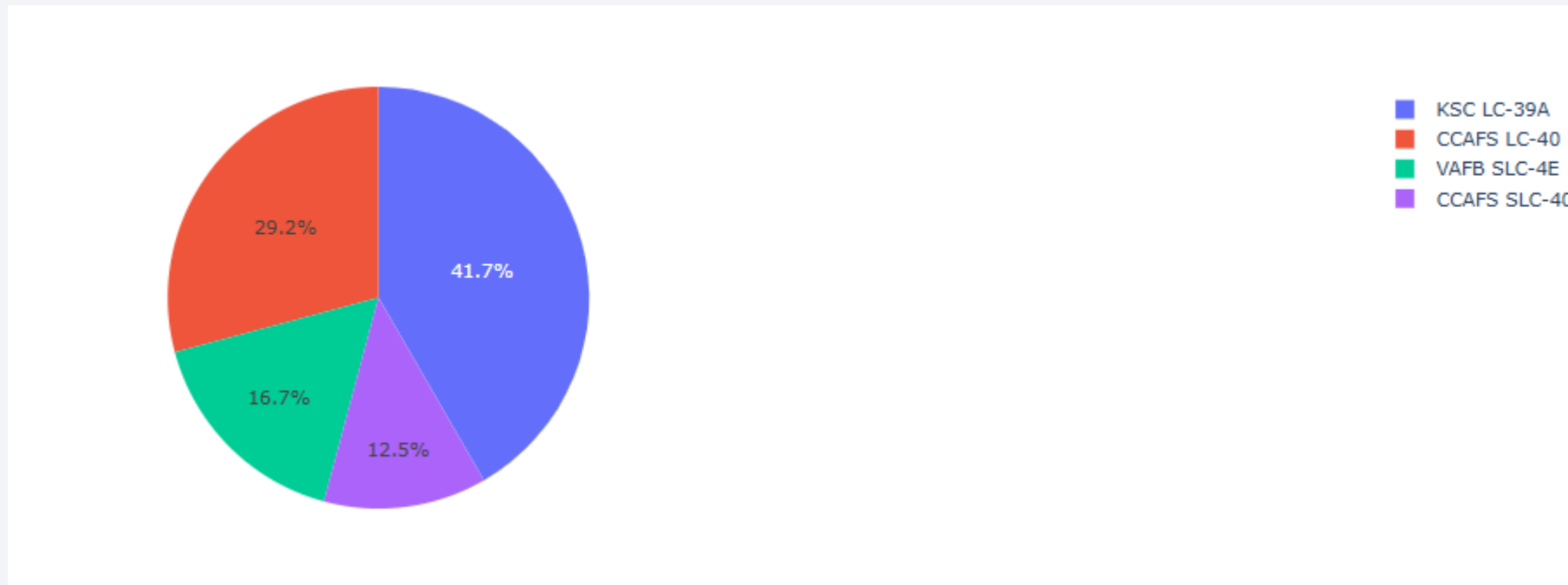


Section 4

Build a Dashboard with Plotly Dash

Launch Success Count for All Sites

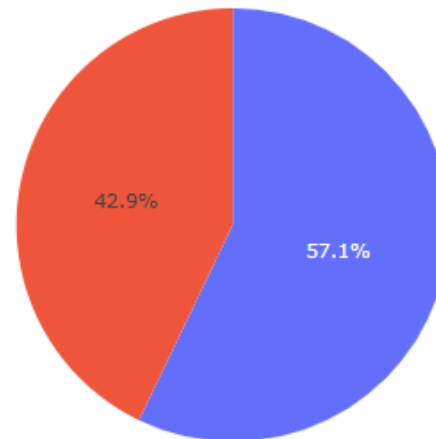
- It shows the success rate per launch site. It is in the percentage.



Launch Site with Highest Success Rate

- The site has success rate of 42.9%. Red shows success and blue for failed launches

Success Rate for site CCAFS SLC-40



<Dashboard Screenshot 3>

- The booster FT and payload ranging 2k-5k has the highest success rate



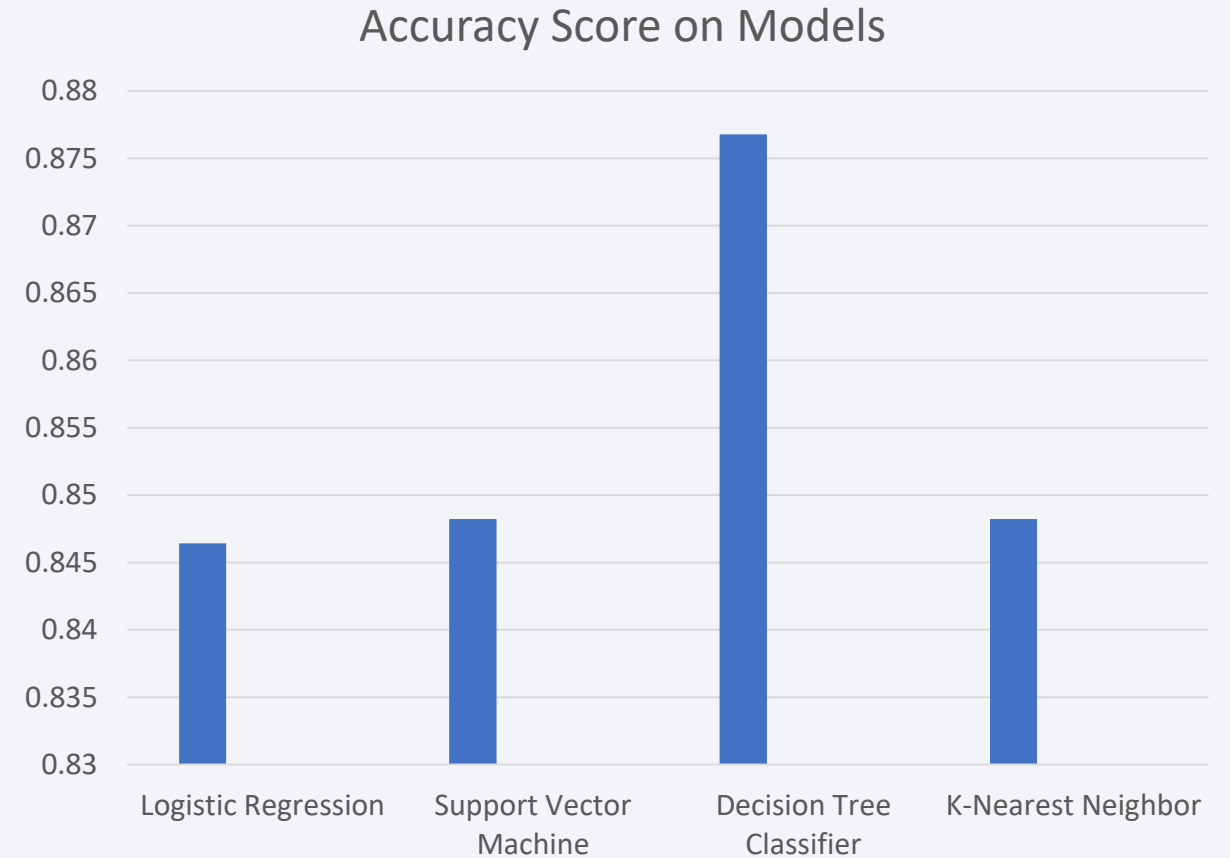


Section 5

Predictive Analysis (Classification)

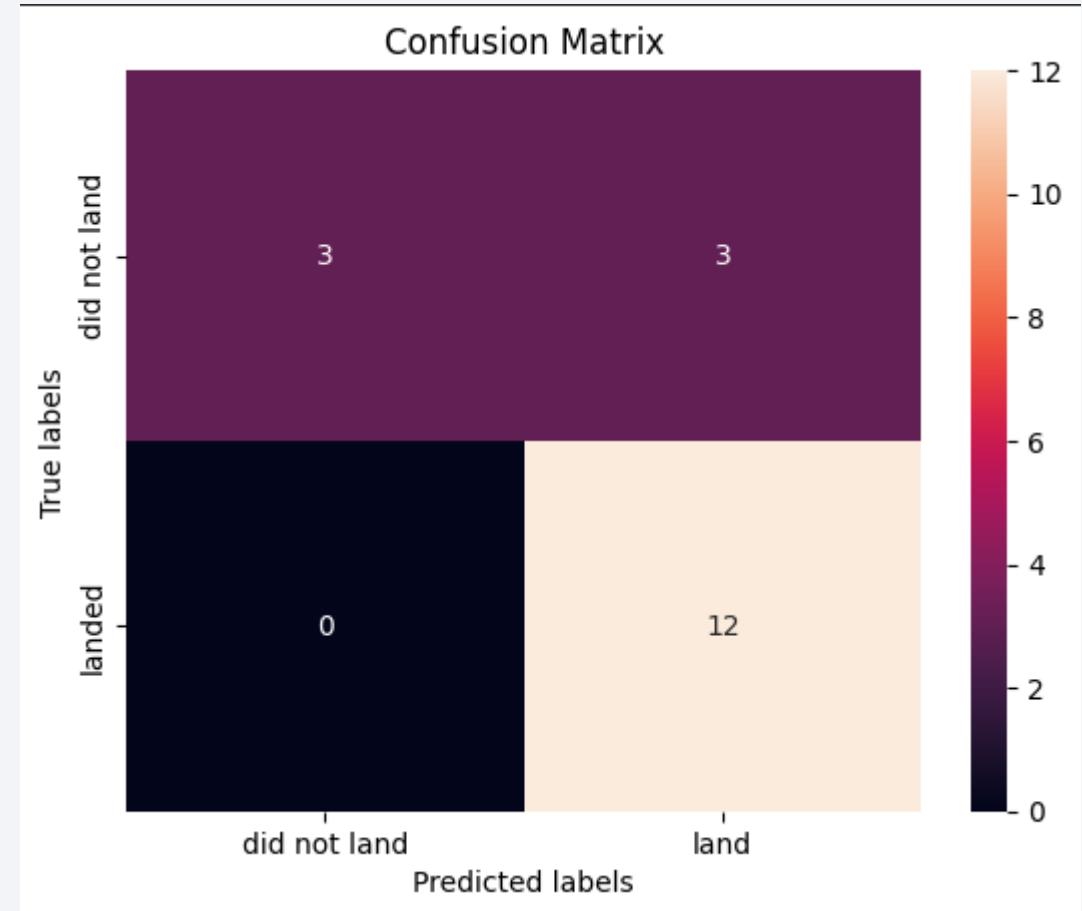
Classification Accuracy

- The Decision Tree model has the highest accuracy of .877



Confusion Matrix

- The model almost predicted all the outcome except in predicting false land.



Conclusions

- **Project Accomplishments**

- Completed an end-to-end data science workflow.
- Developed an interactive dashboard and predictive model.

- **Key Findings**

- Landing success increased over time.
- Launch site, payload mass, and orbit type significantly influenced landing outcomes.

- **Machine Learning**

- Evaluated multiple classification models.
- Selected Decision Tree Classifier as best-performing model through hyperparameter tuning and cross-validation.

- **Final Takeaway**

- Historical SpaceX launch data can be effectively used to predict Falcon 9 first-stage landing success and support data-driven decision-making.

Appendix

```
jupyter-labs-webscraping.ipynb X
ending Prediction > M4 TASK 3: Create a data frame by parsing the launch HTML tables > M4 We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.
Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.12.3
with 1000s, you can choose to write your own logic to parse the HTML tables.

>
extracted_row = 0
#Extract each table
for table_number,table in enumerate(soup.find_all('table','wikitable plainrowheaders collapsible')):
    # get table row
    for rows in table.find_all('tr'):
        #check to see if first table heading is as number corresponding to launch a number
        if rows.th:
            if rows.th.string:
                flight_number=rows.th.string.strip()
                flag=flight_number.isdigit()
            else:
                flag=False
            #get table element
            row=rows.find_all('td')
            #if it is number save cells in a dictionary
            if flag:
                extracted_row += 1
                # Flight Number value
                # TODO: Append the flight_number into launch_dict with key 'Flight No.'
                launch_dict[flight_number].append(flight_number)
                # print(flight_number)

            datatimelist=date_time(row[0])

            # Date value
            # TODO: Append the date into launch_dict with key 'Date'
            date = datatimelist[0].strftime('%Y-%m-%d')
            launch_dict[flight_number].append(date)

[111] ✓ 00s MagicPython
```

```
jupyter-labs-eda-sql-coursera_sqllite.ipynb
C:\Users\User> Documents > Coursera > Data Science > Course 8 - Capstone Project > Module 2 > jupyter-labs-eda-sql-coursera_sqllite.ipynb > M4 Tasks > M4 Task 5
Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.12.3

Task 4

Display average payload mass carried by booster version F9 v1.1

%%sql
select Booster_Version, avg(PAYLOAD_MASS_KG) as 'average payload mass'
from SPACEXTABLE
where Booster_Version = 'F9 v1.1'

[37] ✓ 00s MagicPython
... * sqllite:///my_data1.db
Done.

...
Booster_Version  average payload mass
F9 v1.1          2928.4

Task 5

List the date when the first succesful landing outcome in ground pad was acheived.

Hint:Use min function

[38] ✓ 00s MagicPython
```

```
jupyter-labs-webscraping.ipynb X
ending Prediction > M4 TASK 3: Create a data frame by parsing the launch HTML tables > M4 We can now export it to a CSV for the next section, but to make the answers consistent and in case you have difficulties finishing this lab.
Generate + Code + Markdown | Run All Restart Clear All Outputs Jupyter Variables Outline Python 3.12.3
with 1000s, you can choose to write your own logic to parse the HTML tables.

df= pd.DataFrame({ key:pd.Series(value) for key, value in launch_dict.items() })
df.head(10)

[113] ✓ 00s MagicPython

...
Flight No. Launch site Payload Payload mass Orbit Customer Launch outcome Version Booster Booster landing Date Time
0 1 CCAFS Dragon Spacecraft Qualification Unit 0 orbit customer Success F9 v1.07B0003.18 Failure 4 June 2010 18:45
1 2 CCAFS Dragon 0 orbit customer Success F9 v1.07B0004.18 Failure 8 December 2010 15:43
2 3 CCAFS Dragon 525 kg orbit customer Success F9 v1.07B0005.18 No attempt 22 May 2012 07:44
3 4 CCAFS SpaceX CRS-1 4,700 kg orbit customer Success F9 v1.07B0006.18 No attempt 8 October 2012 00:35
4 5 CCAFS SpaceX CRS-2 4,877 kg orbit customer Success F9 v1.07B0007.18 No attempt 1 March 2013 15:10
5 6 VAFB CASSIOPE 500 kg orbit customer Success F9 v1.17B10038 Uncontrolled 29 September 2013 16:00
6 7 CCAFS SES-8 3,170 kg orbit customer Success F9 v1.1 No attempt 3 December 2013 22:41
7 8 CCAFS Thaicom 6 3,325 kg orbit customer Success F9 v1.1 No attempt 6 January 2014 22:06
8 9 Cape Canaveral SpaceX CRS-3 2,296 kg orbit customer Success F9 v1.1 Controlled 18 April 2014 19:25
9 10 Cape Canaveral Orbcomm-OG2 1,316 kg orbit customer Success F9 v1.1 Controlled 14 July 2014 15:15

0 0 0 Connect Live Share Spaces: 2 Sign In Cell 51 of 58 0:0 Go Live Quokka
```

Create a k nearest neighbors object then create a `GridSearchCV` object `knn_cv` with `cv = 10`

```
knn_parameters = {'n_neighbors': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                  'algorithm': ['auto', 'ball_tree', 'kd_tree', 'brute'],
                  'p': [1,2]}

KNN = KNeighborsClassifier()
knn_cv = GridSearchCV(estimator=KNN, param_grid=knn_parameters, cv=10, verbose=2)
knn_cv.fit(X_train, Y_train)
```

```
Fitting 10 folds for each of 80 candidates, totalling 800 fits
[CV] END .....algorithm=auto, n_neighbors=1, p=1; total time= 3.1s
[CV] END .....algorithm=auto, n_neighbors=1, p=1; total time= 0.0s
[CV] END .....algorithm=auto, n_neighbors=1, p=1; total time= 0.0s
[CV] END .....algorithm=auto, n_neighbors=1, p=1; total time= 0.0s
[CV] END .....algorithm=auto, n_neighbors=1, p=1; total time= 0.0s
```

Appendix

```
SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb
> Users > User > Documents > Coursera > Data Science > Course 8 - Capstone Project > Module 4 > SpaceX-Machine-Learning-Prediction-Part-5-v1.ipynb > M4 Space X Falcon 9 First Stage Landing Prediction > M4 Author
Generate + Code + Markdown + Interrupt + Restart + Clear All Outputs + Go To + Jupyter Variables + Outline + Python 3.12.3
Find the method performs best:

• The Decision Tree performs best with accuracy score of 87.679%.

forest = RandomForestClassifier()
forest_parameters = {
    'n_estimators': [100, 200, 300, 400, 500],
    'max_depth': [None, 5, 10, 20, 30, 40],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 8],
    'max_features': ['sqrt', 'log2']
}
forest_cv = GridSearchCV(estimator=forest, param_grid=forest_parameters, cv=10, verbose=2)
forest_cv.fit(X_train, Y_train)
```

```
%%sql
select Date,
case substr(Date, 6, 2)
WHEN '01' THEN 'January'
WHEN '02' THEN 'February'
WHEN '03' THEN 'March'
WHEN '04' THEN 'April'
WHEN '05' THEN 'May'
WHEN '06' THEN 'June'
WHEN '07' THEN 'July'
WHEN '08' THEN 'August'
WHEN '09' THEN 'September'
WHEN '10' THEN 'October'
WHEN '11' THEN 'November'
WHEN '12' THEN 'December'
end as Month,
Landing_Outcome, Booster_Version, Launch_Site
from SPACEXTABLE
where strftime('%Y', Date) = '2015' and Landing_Outcome = 'Failure (drone ship)'
```

```
xg_parameters = {
    'n_estimators': [100, 200, 300, 400],
    'max_depth': [3, 5, 7, 9],
    'learning_rate': [0.01, 0.1, 0.2],
    'subsample': [0.8, 1.0],
    'colsample_bytree': [0.8, 1.0]
}
xg = XGBClassifier()
xg_cv = GridSearchCV(estimator=xg, param_grid=xg_parameters, cv=10, verbose=3)
xg_cv.fit(X_train, Y_train)

Fitting 10 folds for each of 192 candidates, totalling 1920 fits
[CV 1/10] END colsample_bytree=0.8, learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.8; score=1.000 total time= 0.0s
[CV 2/10] END colsample_bytree=0.8, learning_rate=0.01, max_depth=3, n_estimators=100, subsample=0.8; score=0.750 total time= 0.0s
[CV 3/10] END colsample bytree=0.8, learning rate=0.01, max depth=3, n estimators=100, subsample=0.8; score=0.857 total time= 0.0s
```

Thank you!

